

Книга: MicroPython за VK_RA4M2 чрез демонстрационни примери

Цел

Тази книга е направена върху реално наличния пакет от 54 примерни .py файла за VK_RA4M2.

Тук целта не е само да се изброят файловете.

Целта е да се покрият всички глави, теми и примери от плана, без пропуски и без измислени ресурси.

Затова книгата е организирана в три слоя:

1. Ресурсна карта на борда и порта.
2. Матрица за пълност спрямо учебния план.
3. Подробни глави с кодови откъси и препратки към всички реални сорс файлове.

Как да се ползва книгата

Книгата се чете заедно със сорс файловете в examples.

Препоръчителният ритъм е:

1. Прочитате теорията в съответната глава.
2. • тартирате показания минипример.
3. , тваряте пълния .py файл.
4. Прегледайте ресурсния header в началото му.
5. Променете само по една стойност и набл. „давате ефекта.

Когато примерът съдържа предпазен флаг като DO_WRITE, DO_BOOTLOADER, DO_LIGHTSLEEP, DO_DEEPSLEEP или SET_DEMO_TIME, започвайте с безопасната стойност.

Съдържание

Тази секция дава бърз преглед на всички части и глави в книгата.

Част I: Първи стъпки с борда и с MicroPython

- глава 1. Първа програма
- глава 2. Първи асинхронни примери

Част II: Езикови основи на MicroPython

- глава 3. Променливи и константи
- глава 4. , ператори, условия и цикли

- [...лава 5. Enums в MicroPython](#)
- [...лава 6. Масиви и таблици](#)
- [...лава 7. • труктури в MicroPython](#)
- [...лава 8. †казатели и еквивалентите им в MicroPython](#)
- [...лава 9. ‡ункции, callbacks и таблици от функции](#)

Част III: Цифрови и аналогови входове и изходи

- [...лава 10. Цифров изход](#)
- [...лава 11. Цифров вход](#)
- [...лава 12. ^налогов вход и ADC](#)
- [...лава 13. PWM](#)

Част IV: Таймери, прекъсвания и време

- [...лава 14. Закъснения, хардуерни таймери и Timer\(-1\)](#)
- [...лава 15. Прекъсвания](#)
- [...лава 16. %ебаунс на бутон](#)
- [...лава 17. RTC](#)

Част V: Сериен интерфейс

- [...лава 18. I2C master и runtime избор на пинове](#)
- [...лава 19. SoftI2C и I2CTarget](#)
- [...лава 20. Hardware SPI и SoftSPI](#)

Част VI: TouchPad, памет и системни функции

- [...лава 21. TouchPad: базово четене](#)
- [...лава 22. TouchPad: диагностика, cached API и асинхронно следене](#)
- [...лава 23. renesas.Flash, /flash и dataflash](#)
- [...лава 24. †правление на паметта в MicroPython](#)
- [...лава 25. machine модулът и порт-специфично поведение](#)

Част VII: Индикация и входни матрици

- [...лава 26. %инамична 7-сегментна индикация и матрично сканиране](#)

Част VIII: FSM като начин на мислене

- [...лава 27. Минимална FSM и таблична FSM](#)
- [...лава 28. FSM с timeout](#)
- [...лава 29. Conditions vs Events](#)
- [...лава 30. Guard функции](#)
- [...лава 31. Приоритети и event queue](#)
- [...лава 32. Tickless timers и sleep идея](#)

Част IX: Реален пример

- ...лава 33. Управление на осветление и вентилация за баня
- ...лава 34. %екомпозиция на две FSM

Част X: UART комуникация

- ...лава 35. Шазова UART комуникация

Част XI: asyncio като учебна секция

- ...лава 36. , снови на asyncio
- ...лава 37. asyncio Event синхронизация

Ресурсна карта на VK_RA4M2

€нигата и примерите приемат следната карта на ресурси:

- LED: 1 брой -> LED1 = P204
- Шутон: 1 брой -> SW1 = P400
- USB device пинове: USBDP = P914, USBDM = P915, USB_VBUS = P407
- ADC вън < ни входове: 13 броя -> P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500
- ADC вътре < ни източници: 3 броя -> ADC.CORE_TEMP, ADC.CORE_VREF, ADC.VREF
- DAC изходи: 2 броя -> P014, P015
- PWM изходи: 14 броя -> P107, P106, P105, P104, P113, P114, P112, P115, P608, P409, P408, P600, P304, P303
- UART: 4 инстанции -> UART(0), UART(2), UART(7), UART(9)
- I2C master: 2 инстанции -> I2C(0)=P400/P401, I2C(1)=P100/P101
- I2CTarget: 2 инстанции -> I2CTarget(0)=P400/P401, I2CTarget(1)=P100/P101
- SPI: 1 канал -> CS=P103, SCK=P102, MISO=P100, MOSI=P101
- SoftI2C: наличен
- SoftSPI: наличен
- TouchPad входове: 12 броя -> P205, P206, P407, P408, P409, P410, P411, P412, P413, P414, P415, P708
- CTSU специален пин: P207 = TSCAP
- €ардуерни таймери: 2 броя -> Timer(1), Timer(2)
- •офтуерен таймер: Timer(-1)
- RTC: 1 брой
- •ътре < на файлова система: /flash
- renesas.Flash: наличен block device
- dataflash: 8 KB

- аf ни уточнения:
- SPI тук се разглежда като 1 канал.
- Timer(-1) тук се разглежда като софтуерен таймер.
- SW1 е върху P400, което влиза в конфликт с I2C(0).SCL.
- LED1 е active-low: логическа 0 го светва, логическа 1 го гаси.
- Pin.PULL_UP се ползва в курса; Pin.PULL_DOWN не се разглежда като наличен ресурс в тези уроци.
- lightsleep и deepsleep са показани предпазливо.

Матрица за пълност спрямо плана

Това е контролният списък, по който книгата е проверена.

- Примерна програма -> examples/20_language_basics/01_sample_program.py
- Променливи и константи -> examples/20_language_basics/02_variables_and_constants.py
- Аритметични оператори -> examples/20_language_basics/03_operators_conditions_loops.py
- Логически оператори -> examples/20_language_basics/03_operators_conditions_loops.py
- if / if-else / if-elif-else -> examples/20_language_basics/03_operators_conditions_loops.py
- goto в MicroPython липсва -> обяснено в главата за управление на изпълнението
- switch-case в MicroPython липсва -> обяснено в главата за управление на изпълнението
- while -> examples/20_language_basics/03_operators_conditions_loops.py
- for -> examples/20_language_basics/03_operators_conditions_loops.py
- do-while еквивалент -> examples/20_language_basics/03_operators_conditions_loops.py
- Цифров изход -> examples/21_digital_io/01_digital_output.py, examples/01_blink_async.py
- Цифров вход -> examples/21_digital_io/02_digital_input.py
- Аналогов вход -> examples/22_analog/01_analog_input_adc.py
- Какво е ADC и как работи -> обяснено в главата за ADC и показано в examples/22_analog/01_analog_input_adc.py
- Измерване на ADC.read() с time.ticks_us() -> examples/23_timing/07_adc_timing_measure.py
- PWM изход -> examples/22_analog/02_pwm_output.py, examples/03_pwm_ab_same_channel.py
- Enums -> examples/20_language_basics/04_enums.py
- Масив -> examples/20_language_basics/05_arrays.py
- Масив от масиви -> examples/20_language_basics/05_arrays.py
- Структури -> examples/20_language_basics/06_structures.py
- Структури от структури -> examples/20_language_basics/06_structures.py

- Масив от структури -> `examples/20_language_basics/06_structures.py`
- Указатели като идея -> `examples/20_language_basics/07_pointers_equivalents.py`
- Указател към променлива/масив/структура -> еквиваленти чрез references и mutable обекти в `examples/20_language_basics/07_pointers_equivalents.py`
- Указател към указател -> обяснено като верига от references и индирекция чрез mutable обекти
- Масив от указатели -> еквивалент чрез списък от функции в `examples/20_language_basics/08_functions.py`
- Управление на паметта в MicroPython -> `examples/28_machine_misc/04_memory_management.py`, `examples/20_language_basics/07_pointers_equivalents.py`, `examples/24_i2c/04_i2ctarget_memory.py`
- `gc.collect()`, `gc.mem_free()`, `gc.mem_alloc()`, `micropython.mem_info()` -> `examples/28_machine_misc/04_memory_management.py`
- `memoryview` без копиране и преизползване на буфер -> `examples/20_language_basics/07_pointers_equivalents.py`, `examples/28_machine_misc/04_memory_management.py`, `examples/24_i2c/04_i2ctarget_memory.py`
- Функция -> `examples/20_language_basics/08_functions.py`
- Входни и изходни параметри -> `examples/20_language_basics/08_functions.py`
- Параметри чрез указател -> еквивалент чрез mutable обект в `examples/20_language_basics/08_functions.py`
- Указател към функция -> callback reference в `examples/20_language_basics/08_functions.py`
- Масив от указатели към функции -> `operation_table` в `examples/20_language_basics/08_functions.py`
- Таймери и закъснения -> `examples/23_timing/01_delays_and_timers.py`
- Софтуерен таймер -> `examples/23_timing/04_software_timer_minus1.py`
- Хардуерен срещу софтуерен таймер -> `examples/23_timing/05_timer_compare_hw_sw.py`
- `time.ticks_us()` и относителни измервания -> `examples/23_timing/07_adc_timing_measure.py`
- Прекъсвания -> `examples/23_timing/02_interrupts.py`
- Измерване на interrupt latency -> `examples/23_timing/08_interrupt_latency_measure.py`
- Приоритети -> `examples/30_fsm/06_priorities.py`
- Забраняване/разрешаване на прекъсвания -> `examples/23_timing/02_interrupts.py`, `examples/28_machine_misc/02_machine_mem_and_bootloader.py`
- Четене на бутон и дебаунс -> `examples/23_timing/03_button_debounce.py`
- Динамична 7-сегментна индикация -> `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`
- Прекъсвания по таймер / времево опресняване -> обяснено в главата за display scan и timer главите

- Матрично сканиране на бутони -> `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`
- FSM идея -> `examples/30_fsm/01_fsm_lamp_button.py`
- Графично и таблично описание на FSM -> обяснено в главата за FSM и показано в `examples/30_fsm/02_fsm_table.py`
- FSM с timeout -> `examples/30_fsm/03_fsm_timeout.py`
- Conditions vs Events -> `examples/30_fsm/04_conditions_vs_events.py`
- Guard функции -> `examples/30_fsm/05_guard_functions.py`
- Реален пример: баня -> `examples/31_case_study/01_bathroom_control.py`
- Декомпозиция на 2 FSM -> `examples/31_case_study/02_two_fsms.py`
- Комуникация между стейт машини -> `examples/31_case_study/02_two_fsms.py`
- Event queue -> `examples/30_fsm/07_event_queue.py`
- Tickless timers + sleep -> `examples/30_fsm/08_tickless_sleep.py`,
`examples/28_machine_misc/03_sleep_modes.py`

Част I: Първи стъпки с борда и с MicroPython

Глава 1. Първа програма

Първата програма трябва да даде веднага видим резултат и да покаже как изглежда MicroPython файл за VK_RA4M2.

Ел, чова идея:

```
from machine import Pin # Импортираме класа Pin, чрез който управляваме GPIO пиновете.
import time # Импортираме модула time, за да правим закъснения между действията.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изходен обект за LED1 и започваме с изгасен LED.

for blink_index in range(3): # Обхождаме три последователни мигания с for цикъл.
    led.value(0) # Светваме LED1 като подадем активното за платката ниво.
    time.sleep_ms(300) # Изчакваме 300 милисекунди, за да се вижда светването.
    led.value(1) # Гасим LED1, за да завършим едно мигане.
    time.sleep_ms(300) # Изчакваме още 300 милисекунди преди следващото мигане.
```

Тук се виждат наведнъж:

- Pin
- for
- sleep_ms
- active-low поведението на LED1

† айлове в главата:

- `examples/20_language_basics/01_sample_program.py`

Резюме

- ...лавата показва най-малкия работещ MicroPython пример за платката.
- ‡окусът е върху `Pin`, `sleep_ms()` и active-low поведението на LED1.
- Това е отправната точка за всички следващи и практически уроци.

Самопроверка

1. Защо LED1 светва при `value(0)`?
2. Каква е ролята на `Pin.OUT` и `sleep_ms()` в примера?
3. Как е най-малката промяна, с която бихте проверили, че разбирате кода?

Упражнения

1. Променете броя мигания и дължината на паузата и наблюдавайте резултата.
2. •аправете различни времена за светнато и изгасено състояние.
3. •знесете броя мигания и закъснението в именувани константи.

Ако не работи

- импотом: LED1 не мига. Проверка: използвайте `Pin("LED1", Pin.OUT)` и имайте предвид active-low логиката.
- импотом: ефектът е невидим. Проверка: вероятно паузите са твърде кратки или цикълът не се изпълнява както очаквате.

Нататък

- лед това мини към следващата глава, 'Първи асинхронни примери', за да надгради текущата идея в естествената следваща стъпка.

Глава 2. Първи асинхронни примери

Преди да влезем в детайлите на езика, полезно е да се види как изглежда да неблокираш мислене.

Минипример:

```
import asyncio # Импортираме asyncio, за да правим неблокиращи задачи в MicroPython.
from machine import Pin # Импортираме класа Pin, за да управляваме LED1 като цифров изход.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изход за LED1 и започваме от изгасено състояние.

async def blink_task(): # Дефинираме асинхронна задача, която ще мига без да блокира event loop-a.
    while True: # Повтаряме мигането безкрайно.
```

```

led.value(0) # Светваме LED1.
await asyncio.sleep_ms(500) # Отстъпваме управлението за 500 ms.
led.value(1) # Гасим LED1.
await asyncio.sleep_ms(500) # Отново отстъпваме управлението за 500 ms.

```

• тори минипример:

```

async def count_seconds_task(): # Създаваме втора задача, която брой секундите.
    seconds = 0 # Началната стойност на брояча е нула секунди.
    while True: # Повтаряме и тази задача безкрайно.
        seconds += 1 # Увеличаваме брояча с една секунда.
        print("Изминали секунди:", seconds) # Показваме колко време е минало от старта
    .
    await asyncio.sleep(1) # Чакаме точно една секунда без блокиране на event loop
-a.

```

Тези примери показват две идеи:

- `asyncio` като кооперативен scheduler
- разделяне на поведението на независими задачи

‡ айлове в главата:

- `examples/01_blink_async.py`
- `examples/02_two_tasks.py`

Резюме

- ...лавата въве *f* да кооперативното мислене с `asyncio`.
- Показва се как две задачи могат да споделят процесора без взаимно блокиране.
- Това е основа за неблокира^Щ и прило^fения върху борда.

Самопроверка

1. Ёакво прави `await` в асинхронната задача?
2. За^Що една `async` задача не бива да работи без да отстъпва управлението?
3. Ёаква е практическата разлика ме^fду две `async` задачи и един блокира^Щ цикъл?

Упражнения

1. %обавете трета `async` задача с различен период.
2. Променете периодите на задачите и набл „давайте как се преплитат.
3. %обавете брояч, който спира демонстрацията след определено време.

Ако не работи

- имптом: ви^fдате поведение само от една задача. Проверка: търсете липсва^Щ `await` или неправилно създаване на задачите.

- импотом: програмата не тръгва асинхронно. Проверка: уверете се, че event loop-ът се стартира по правилния начин.

Нататък

- лед това мини към следващата глава, 'Променливи и константи', за да надгради текущата идея в естествената "следваща стъпка.

Част II: Езикови основи на MicroPython

Глава 3. Променливи и константи

Пример:

```
from micropython import const # Импортираме const, за да дефинираме истински MicroPython константи.
```

```
BOARD_NAME = "VK_RA4M2" # Това е обикновена низова променлива с името на платката.
```

```
LED_ALIAS = "LED1" # Това е още една обикновена променлива, която пази board alias на LED-a.
```

```
BLINK_COUNT = const(3) # Това е константа, която няма да променяме по време на работа.
```

```
BLINK_DELAY_MS = const(200) # Това е константа за закъснение в милисекунди.
```

```
uses_asyncio = True # Булева променлива ни показва дали платката има поддръжка на asyncio.
```

Тук разграничението е просто:

- променлива е нещо, което променяме
- const() е именувана константа, с която описваме числа и рефимии

файл:

- examples/20_language_basics/02_variables_and_constants.py

Резюме

- ...лавата въвежда темата 'Променливи и константи' чрез кратък изпълним MicroPython пример.
- деята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Ёоя е централната идея в 'Променливи и константи'?
2. Ёои конструкции от примера трябва да разпознавате без колебание?
3. Ёак бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Ёобавете втори кратък пример по същата тема с други данни.

3. • върнете техниката от тази глава с материала от предишната глава.

Ако не работи

- импотом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- импотом: логиката е неясна. Проверка: отпечатвайте междупродължителните стойности и следете реда на изпълнение.

Нататък

• Вед това мини към следващата глава, 'Оператори, условия и цикли', за да надградиш текущата идея в естествената "следваща стъпка."

Глава 4. Оператори, условия и цикли

Тази глава покрива аритметика, сравнения, логика и контрол на потока.

Пример:

```
left_value = 7 # Първата числова променлива участва в аритметичните операции.
right_value = 3 # Втората числова променлива е вторият операнд.

sum_value = left_value + right_value # Събираме двете стойности с оператора плюс.
logic_result = (left_value > right_value) and (left_value != right_value) # Комбинирам
е логически оператори.

if left_value < right_value: # Проверяваме първото условие.
    print("По-малка") # Този ред би се изпълнил, ако първото условие е вярно.
elif left_value == right_value: # Ако първото условие е невярно, проверяваме второто.
    print("Равна") # Този ред би се изпълнил при равенство.
else: # Ако нито едно от горните условия не е изпълнено, влизаме в else клона.
    print("По-голяма") # Тук попадаме с текущите примерни стойности.
```

Циклите са показани и в трите практически форми:

```
for index in range(3): # Изпълняваме цикъл точно три пъти.
    print(index) # Печатаме текущата стойност на брояча.

counter = 0 # Създаваме брояч за while цикъла.
while counter < 3: # Повтаряме докато броячът е по-малък от три.
    print(counter) # Печатаме текущата стойност на брояча.
    counter += 1 # Увеличаваме брояча, за да не останем в безкраен цикъл.

attempt = 0 # Създаваме брояч за демонстрацията.
while True: # Влизаме в безкраен цикъл, който ще прекъснем с break.
    print(attempt) # Печатаме текущата стойност, като това винаги се случва поне веднъж.
```

```

attempt += 1 # Увеличаваме брояча след изпълнението на тялото.
if attempt >= 2: # Проверяваме условието за прекратяване след тялото на цикъла.
    break # Напускаме цикъла, което прави конструкцията еквивалент на do-while.

```

• аfни уточнения за начинаеŽи:

- goto няма в Python и в MicroPython
- switch-case няма в класическия MicroPython синтаксис на тези примери
- еквивалентите са if/elif/else, таблици, break, continue, return
- do-while се показва като while True + break

‡айл:

- examples/20_language_basics/03_operators_conditions_loops.py

Резюме

- ...лавата въвеf да темата ‘ , ператори, условия и цикли’ чрез кратък изпълним MicroPython пример.
- •деята е показана в чист синтаксис, без да се губи връзката с бъдеŽите хардуерни примери.
- Това е езиков строителен блок за по-слоfните глави нататък.

Самопроверка

1. Ёоя е централната идея в ‘ , ператори, условия и цикли’?
2. Ёои конструкции от примера трябва да разпознавате без колебание?
3. Ёак бихте прилоfили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. %обавете втори кратък пример по съŽата тема с други данни.
3. •върfете техниката от тази глава с материала от преди <ната глава.

Ако не работи

- •имптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- •имптом: логиката е неясна. Проверка: отпечатайте меfдинните стойности и следете реда на изпълнение.

Нататък

- лед това мини към следваŽата глава, ‘Enums в MicroPython’, за да надгради <текуŽата идея в естествената “следваŽа стъпка.

Глава 5. Enums в MicroPython

Пример:

```
from micropython import const # Импортираме const, за да правим именувани константи като прост MicroPython еквивалент на enum.
```

```
STATE_OFF = const(0) # Тази константа означава състояние "изключено".
```

```
STATE_ON = const(1) # Тази константа означава състояние "включено".
```

```
STATE_BLINK = const(2) # Тази константа означава състояние "мигане".
```

Това е правилният стил за тези уроци:

- именувани константи
- таблица за превод към четимо име

† айл:

- `examples/20_language_basics/04_enums.py`

Резюме

- ...лавата въве *f* да темата 'Enums в MicroPython' чрез кратък изпълним MicroPython пример.
- •деята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Ёоя е централната идея в 'Enums в MicroPython'?
2. Ёои конструкции от примера трябва да разпознавате без колебание?
3. Ёак бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. %обавете втори кратък пример по същата тема с други данни.
3. •върнете техниката от тази глава с материала от предишната глава.

Ако не работи

- •имптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- •имптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

- лед това мини към следващата глава, 'Масиви и таблици', за да надградите текущата идея в естествената "следваща стъпка."

Глава 6. Масиви и таблици

Пример:

```

from array import array # Импортираме array, за да покажем по-компактен числов масив от list.

adc_samples = [1200, 1215, 1198, 1222] # Това е обикновен list, който играе роля на масив от измервания.

led_table = [["LED1", "P204"], ["SW1", "P400"]] # Това е вложен масив, тоест таблица от редове и колони.

pwm_duty_values = array("H", [0, 16384, 32768, 49152, 65535]) # Тук пазим 16-битови стойности в компактен масив.

raw_bytes = bytearray([0x10, 0x20, 0x30, 0x40]) # bytearray е удобен за сурови байтове и комуникационни буфери.

```

Тук има четири различни вида колекции:

- list
- вложен list, тоест масив от масиви
- array
- bytearray

Най-важно:

- examples/20_language_basics/05_arrays.py

Резюме

- Главната въвеждаща темата 'Масиви и таблици' чрез кратък изпълним MicroPython пример.
- Главната е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Каква е централната идея в 'Масиви и таблици'?
2. Какви конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Набелязвайте втори кратък пример по същата тема с други данни.
3. Приложете техниката от тази глава с материала от предишната глава.

Ако не работи

- симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

• лед това мини към следващата глава, '• структури в MicroPython', за да надгради текущата идея в естествената "следваща стъпка.

Глава 7. Структури в MicroPython

• място `struct` от C, в MicroPython се ползват:

- `dict`
- `namedtuple`
- клас
- списък от такива обекти

Пример:

```
from collections import namedtuple # Импортираме namedtuple като лек и четим заместител на проста структура.
```

```
SensorRecord = namedtuple("SensorRecord", ("name", "pin", "unit")) # Описваме поле по поле една проста структура.
```

```
light_sensor = {"name": "Light", "adc_pin": "P000", "limits": {"dark": 800, "bright": 3000}} # dict е най-честият MicroPython заместител на структура.
```

```
devices = [{"name": "LED1", "pin": "P204", "kind": "output"}] # Тук правим списък от структури, както в C бихме имали масив от struct.
```

Тук има и:

- структура от структури чрез вложен `dict`
- масив от структури чрез списък от `dict`

џайл:

- `examples/20_language_basics/06_structures.py`

Резюме

- ...лавата въвежда темата '• структури в MicroPython' чрез кратък изпълним MicroPython пример.
- • деята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Ёоя е централната идея в '• структури в MicroPython'?
2. Ёои конструкции от примера трябва да разпознавате без колебание?
3. Ёак бихте приложили или тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.

3. • върнете техниката от тази глава с материала от предишната глава.

Ако не работи

- импотом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- импотом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

• след това мини към следващата глава, 'Указатели и еквивалентите им в MicroPython', за да надградиш текущата идея в естествената "следваща стъпка."

Глава 8. Указатели и еквивалентите им в MicroPython

Тази глава е умело преведена към MicroPython, а не към C синтаксис.

Пример:

```
adc_list = [100, 200, 300] # Създаваме изменяем обект, който ще играе ролята на споделени данни.  
alias_list = adc_list # Тук не се копира list, а и двете имена сочат към един и същ обект.  
alias_list[1] = 999 # Промяната през второто име се вижда и през първото име.  
  
tx_buffer = bytearray([1, 2, 3, 4]) # bytearray е полезен за буфери, които се редактират "на място".  
tx_view = memoryview(tx_buffer) # memoryview дава прозорец към същите байтове без копияне.  
tx_view[2] = 77 # Промяната през memoryview променя и оригиналния буфер.
```

Тук се показват:

- две имена към един и същ обект
- промяна на буфер 'на място'
- memoryview като евтин прозорец към същите данни
- ниско ниво чрез machine.mem8, mem16, mem32

Файлове:

- examples/20_language_basics/07_pointers_equivalents.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py

Резюме

- ...главата въвежда темата 'Указатели и еквивалентите им в MicroPython' чрез кратък изпълним MicroPython пример.
- деята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.

- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Ȑоя е централната идея в ‘Ȑказатели и еквивалентите им в MicroPython’?
2. Ȑои конструкции от примера трябва да разпознавате без колебание?
3. Ȑак бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. • върнете техниката от тази глава с материала от предишната глава.

Ако не работи

- импотом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- импотом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

- Вед това мини към следващата глава, ‘Ȑункции, callbacks и таблици от функции’, за да надградите текущата идея в естествената следваща стъпка.

Глава 9. Функции, callbacks и таблици от функции

Пример:

```
def add_values(left_value, right_value): # Дефинираме функция, която получава два входни параметъра.
    return left_value + right_value # Връщаме резултата като изходна стойност.

def run_callback(callback, value): # Дефинираме функция, която приема друга функция като параметър.
    callback(value) # Извикваме подадената функция с подадената стойност.

def square(value): # Това е първата функция за таблицата от функции.
    return value * value # Връщаме квадрата на аргумента.

operation_table = [square] # Това е списък от функции, тоест таблица от извикваеми обекти.
```

Този глава покрива:

- входни параметри
- връщане на стойности
- multiple return values
- mutable параметри вместо ‘предаване с указател’

- функция като стойност
- списък от функции като таблица от 'function pointers'

† айл:

- `examples/20_language_basics/08_functions.py`

Резюме

- ...лавата въве *f* да темата '† функции, callbacks и таблици от функции' чрез кратък изпълним MicroPython пример.
- • деята е показана в чист синтаксис, без да се губи връзката с бъдежните хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Ёоя е централната идея в '† функции, callbacks и таблици от функции'?
2. Ёои конструкции от примера трябва да разпознавате без колебание?
3. Ёак бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. %обавете втори кратък пример по същата тема с други данни.
3. • върнете техниката от тази глава с материала от предишната глава.

Ако не работи

- импотом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- импотом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

- Вед това мини към следващата глава, 'Цифров изход', за да надградите текущата идея в естествената "следваща стъпка.

Част III: Цифрови и аналогови входове и изходи

Глава 10. Цифров изход

Пример:

```
from machine import Pin # Импортираме класа Pin, за да използваме LED1 като цифров изход.

import time # Импортираме time, за да правим видими паузи между смените на логическото ниво.

led = Pin("LED1", Pin.OUT, value=1) # Настройваме LED1 като изход и го оставяме изгасено в началото.
```

```
led.value(0) # Подаваме логическа нула на изхода.  
time.sleep_ms(700) # Изчакваме 700 ms, за да се види светването.  
led.value(1) # Подаваме логическа единица на изхода.
```

Тук се обясняват:

- логическа 0
- логическа 1
- защо при active-low LED те не означават директно 'изкл „ чено' и 'вкл „ чено'
- че допустимият ток и товароспособността се гледат по hardware manual и схемата на борда

џайлове:

- examples/21_digital_io/01_digital_output.py
- examples/01_blink_async.py

Резюме

- ...лавата показва как темата 'Цифров изход' се връзва към реален хардуерен ресурс на платката.
- , сновният фокус е върху правилния API, настройката на пиновете и набл „даемия резултат.
- Тук езиковият синтаксис вече слупи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Ёой хардуерен ресурс или API е централният в 'Цифров изход'?
2. Ёоя стойност, реfим или параметър влияе най-много на поведението?
3. Ёак бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и набл „давайте как се променя физическият ефект.
2. %обавете кратък печат за диагностика на текущо състояние или измерване.
3. Ёомбинируйте темата с LED1 или SW1, ако това е прилоfимо.

Ако не работи

- • имптом: няма хардуерна реакция. Проверка: сверете alias/pin, реfима и активното логическо ниво.
- • имптом: реакцията е обърната или < умна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

- лед това мини към следващата глава, 'Цифров вход', за да надгради < текущата идея в естествената " следващата стъпка.

Глава 11. Цифров вход

Пример:

```
from machine import Pin # Импортираме Pin, за да четем бутона SW1 като цифров вход.  
  
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме бутона като вход с вътрешен pull-up резистор.  
  
print(button.value()) # Вземаме текущата сурова логическа стойност на входа.
```

Тук се обясняват:

- входно ниво
- активен нисък бутон
- вътрешен PULL_UP
- защо PULL_DOWN не е част от тези уроци за този порт

џаил:

- `examples/21_digital_io/02_digital_input.py`

Резюме

- ...лавата показва как темата 'Цифров вход' се връзва към реален хардуерен ресурс на платката.
- , сновният фокус е върху правилния API, настройката на пиновете и набл „даемия резултат.
- Тук езиковият синтаксис вече слуф и на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Ёой хардуерен ресурс или API е централният в 'Цифров вход'?
2. Ёоя стойност, реf им или параметър влияе най-много на поведението?
3. Ёак бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и набл „давайте как се променя физическият ефект.
2. %обавете кратък печат за диагностика на текущото състояние или измерване.
3. Ёомбинируйте темата с LED1 или SW1, ако това е прилоf имо.

Ако не работи

- • имптом: няма хардуерна реакция. Проверка: сверете alias/pin, реf има и активното логическо ниво.
- • имптом: реакцията е обърната или < умна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

- лед това мини към следващата глава, 'налогов вход и ADC', за да надгради < текущата идея в естествената "следващата стъпка.

Глава 12. Аналогов вход и ADC

Пример:

```
from machine import ADC # Импортираме класа ADC, чрез който се четат аналоговите канал и.

temperature_adc = ADC(ADC.CORE_TEMP) # Създаваме ADC обект за вътрешния температурен канал.

vref_adc = ADC(ADC.CORE_VREF) # Създаваме ADC обект за вътрешното опорно напрежение.

print(temperature_adc.read()) # Четем суровата ADC стойност на вътрешния температурен канал.

print(vref_adc.read()) # Четем суровата ADC стойност на вътрешното опорно напрежение.
```

Тук се учат:

- какво е ADC
- как напрежението става число
- разликата между сурово четене и `read_u16()`
- защо вътрешните канали са добър старт
- а също и уточнение за RA4M2:
- Този MCU е с ADC12, тоест хардуерната резолуция тук е 12 bit, а поддържаните режими са 12 / 10 / 8 bit.
- текущия порт може да се избира режим още при създаване на обекта: `ADC(pin_or_channel, bits=8)`, `ADC(..., bits=10)`, `ADC(..., bits=12)`.
- астройката за bits е глобална за ADC периферията, а не отделна за всеки ADC обект.

Ератък пример:

```
from machine import ADC # Импортираме ADC за избор на резолюция при създаване на обекта.

adc_fast = ADC("P000", bits=8) # Създаваме ADC обект с 8-битова резолюция за бързо четене.

adc_balanced = ADC("P001", bits=10) # Създаваме ADC обект с 10-битова резолюция за баланс бързина/точност.

adc_precise = ADC("P002", bits=12) # Създаваме ADC обект с 12-битова резолюция за максимална точност.

print(adc_fast.read(), adc_balanced.read(), adc_precise.read()) # Четем и трите канала и показваме резултатите.
```

Практическа бележка за времето на преобразуване при текущите тактове на VK_RA4M2:

- при минимално валидно семплиране и $ADCLK = 50 \text{ MHz}$ самото A/D преобразуване е приблизително 0.31 us за 8 bit, 0.35 us за 10 bit, 0.39 us за 12 bit

- при по-дълго семплиране, например около reset/default настройката на ADSSTR, стойностите са приблизително 0.41 us, 0.45 us, 0.49 us
- това е време на хардуерния ADC, а не пълното време на извикването ADC.read() от Python
- ако искате да мерите целия път през MicroPython, използвайте time.ticks_us() около самото извикване
- за практическо измерване на пълното време на ADC.read() виж examples/23_timing/07_adc_timing_measure.py, където се мери последователно в 8 / 10 / 12 bit
- аличните външни ADC пинове в този курс са:

P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500

файл:

- examples/22_analog/01_analog_input_adc.py
- examples/23_timing/07_adc_timing_measure.py

Резюме

- ...главата показва как темата 'външен вход и ADC' се връзва към реален хардуерен ресурс на платката.
- , сновният фокус е върху правилния API, настройката на пиновете и на бл „даемия резултат.
- Тук езиковият синтаксис вече служи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Кой хардуерен ресурс или API е централният в 'външен вход и ADC'?
2. Коя стойност, рефим или параметър влияе най-много на поведението?
3. Как бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и на бл „давайте как се променя физическият ефект.
2. Добавете кратък печат за диагностика на текущото състояние или измерване.
3. Комбинируйте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- импотом: няма хардуерна реакция. Проверка: сверете alias/pin, рефима и активното логическо ниво.
- импотом: реакцията е обърната или < умна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

- След това мини към следващата глава, 'PWM', за да надградиш текущата идея в естествената "следваща стъпка.

Глава 13. PWM

Šазов пример:

```
from machine import Pin, PWM # Импортираме Pin и PWM, за да генерираме PWM сигнал върху валиден PWM пин.
```

```
pwm = PWM(Pin("P107"), freq=1000, duty=50) # Създаваме PWM обект с честота 1 kHz и duty 50 процента.
```

Разширен пример за обш GPT канал:

```
pwm_a = PWM(Pin("P107"), freq=1000, duty=25) # Създаваме PWM върху изход A с честота 1 kHz и duty 25 процента.
```

```
pwm_b = PWM(Pin("P106"), freq=1000, duty=75) # Създаваме PWM върху изход B със същата честота, но с duty 75 процента.
```

```
pwm_a.freq(2000) # Променяме честотата от страна на изход A, което ще промени и канала B, защото са върху един GPT канал.
```

Тук се вижда, че:

- duty може да е различен
- честотата се споделя от двата изхода на обшия GPT канал
- на този порт PWM е върху GPT периферията и обшият канал налага обша freq() за A/B изходите
- проверката на реалната форма, фронтове и jitter става с осцилоскоп или логически анализатор, а не само по print от Python

џайлове:

- examples/22_analog/02_pwm_output.py
- examples/03_pwm_ab_same_channel.py

Резюме

- ...лавата показва как темата 'PWM' се връзва към реален хардуерен ресурс на платката.
- , сновният фокус е върху правилния API, настройката на пиновете и набл „даемия резултат.
- Тук езиковият синтаксис вече служ и на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Ёой хардуерен ресурс или API е централният в 'PWM'?
2. Ёоя стойност, рефим или параметър влияе най-много на поведението?
3. Ёак бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и набл „давайте как се променя физическият ефект.
2. %обавете кратък печат за диагностика на текущото състояние или измерване.
3. Ёомбинируйте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- имптом: няма хардуерна реакция. Проверка: сверете alias/pin, ре f има и активното логическо ниво.
- имптом: реакцията е обърната или < умна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

- лед това мини към следващата глава, 'Закъснения, хардуерни таймери и Timer(-1)', за да надграти < текущата идея в естествената " следваща стъпка.

Част IV: Таймери, прекъсвания и време

Глава 14. Закъснения, хардуерни таймери и Timer(-1)

Блокиращо закъснение:

```
import time # Импортираме time за блокиращото закъснение.  
time.sleep_ms(300) # Това закъснение блокира изпълнението на останалия код.
```

Хардуерен таймер:

```
from machine import Timer # Импортираме Timer за хардуерния таймер.  
  
timer = Timer(1) # Използваме първия от двата налични machine.Timer канала.  
timer.init(freq=4, callback=timer_callback, hard=False) # Стартираме Timer(1) на 4 Hz  
с callback извън hard IRQ контекст.
```

Софтуерен таймер:

```
from machine import Pin, Timer # Импортираме Pin и Timer за демонстрацията на софтуерн  
ия таймер.  
  
import time # Импортираме time, за да оставим таймера да работи кратко време.  
  
led = Pin("LED1", Pin.OUT) # Вземаме LED1 като видим индикатор за сработване на callba  
ck-a.  
  
soft_timer = Timer(-1) # Създаваме софтуерен таймер чрез идентификатор -1.  
soft_state = {"count": 0} # Пазим броя сработвания в mutable структура.  
started = False # С този флаг ще следим дали таймерът е стартирал успешно.  
  
def soft_timer_callback(timer_object): # Това е функцията, която ще се извиква при вся  
ко сработване на софтуерния таймер.  
    soft_state["count"] = soft_state["count"] + 1 # Увеличаваме брояча при всяко събит  
ие.  
    led.value(0 if led.value() else 1) # Превключваме LED, за да виждаме че callback-ъ  
т се изпълнява.  
  
if not started: # Ако още не сме стартирали таймера, пробваме втория стил с честота.
```

```

try: # Влизаме в защитен блок за втория опит.
    soft_timer.init(freq=4, callback=soft_timer_callback, hard=False) # Пробваме и
нициализация с честота 4 Hz.
    started = True # Отбелязваме, че вторият стил е успял.
    print("Timer(-1) тръгна с freq API.") # Печатаме кой резервен стил е сработил.
except Exception as second_error: # Ако и вторият стил не тръгне, само информираме
потребителя.
    print("freq API също не тръгна:", second_error) # Показваме причината и за вто
рия неуспех.

```

Тук трябва да остане ясно:

- Timer(1) и Timer(2) са хардуерни
- Timer(-1) е софтуерен
- asyncio е още една неблокираща алтернатива

Практическа бележка за time.ticks_us():

- ползвайте time.ticks_us() и time.ticks_diff() за относителни интервали в микросекунди
- в този порт time.ticks_us() идва от хардуерно поддържан mp_hal_ticks_us(), а не от измислен Python брояч
- това е много полезно за сравнение на latency и ADC.read() време, но не е заместител на осцилоскоп при хардуерна прецизност

џайлове:

- examples/23_timing/01_delays_and_timers.py
- examples/23_timing/04_software_timer_minus1.py
- examples/23_timing/05_timer_compare_hw_sw.py
- examples/23_timing/07_adc_timing_measure.py

Резюме

- ...лавата развива темата 'Закъснения, хардуерни таймери и Timer(-1)' около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Ёоя времева или събитийна идея е кл „чова в 'Закъснения, хардуерни таймери и Timer(-1)'?
2. Ёак бихте разграничили удобна демонстрация от надеждно реално поведение?
3. Ёое в тази глава би се повредило най-лесно при твърде блокиращ код?

Упражнения

1. Променете интервал, период или timeout и проследете разликата в поведението.

2. %обавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. •равнете поведението на два различни подхода от главата върху една и съща задача.

Ако не работи

- импотом: поведението е нестабилно във времето. Проверка: търсете дълги блокиражи и участъци и неправилни единици за време.
- импотом: получавате твърде много или твърде малко събития. Проверка: вижте прагове, debounce, периоди и условия за сработване.

Нататък

- вед това мини към следващата глава, 'Прекъсвания', за да надгради текущата идея в естествената "следваща стъпка.

Глава 15. Прекъсвания

Пример:

```
from machine import Pin, disable_irq, enable_irq # Импортираме Pin, disable_irq и enable_irq за демонстрацията.

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме SW1 като вход с pull-up.

def button_handler(pin_object): # Това е функцията, която ще се вика при прекъсване от бутона.
    print("IRQ") # Печатаме текст при всяко прекъсване.

button.irq(handler=button_handler, trigger=Pin.IRQ_FALLING, hard=False) # Регистрираме обработчик за падащ фронт на SW1.
```

Безопасно четене на споделени данни:

```
irq_state = disable_irq() # Временно забраняваме прекъсванията, за да вземем консистентно копие на брояча.

snapshot = irq_state_data["count"] # Копираме брояча, докато прекъсванията са спрени.

enable_irq(irq_state) # Възстановяваме предишното състояние на прекъсванията.
```

Това е преходът към реално събитийно програмиране.

Практическа бележка за измерване на latency:

- може да се направи полезна ориентировъчна оценка чрез два свързани GPIO пина и `time.ticks_us()`
- тази стойност вкл. чва GPIO пътя, IRQ доставката, soft callback scheduling и Python callback overhead
- затова тя не е равна на 'чистата' силициева interrupt latency от datasheet или осцилоскоп
- примерът `examples/23_timing/08_interrupt_latency_measure.py` иска jumper между изходен и входен пин

†айлове:

- `examples/23_timing/02_interrupts.py`
- `examples/23_timing/08_interrupt_latency_measure.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

Резюме

- ...лавата развива темата 'Прекъсвания' около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Ёоя времева или събитийна идея е кл „чова в 'Прекъсвания'?
2. Ёак бихте разграничили удобна демонстрация от наде fдно реално поведение?
3. Ёое в тази глава би се повредило най-лесно при твърде блокира Ź код?

Упражнения

1. Променете интервал, период или timeout и проследете разликата в поведението.
2. %обавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. •равнете поведението на два различни подхода от главата върху една и съ Źа задача.

Ако не работи

- •имптом: поведението е нестабилно във времето. Проверка: търсете дълги блокира Źи участъци и неправилни единици за време.
- •имптом: получавате твърде много или твърде малко събития. Проверка: ви fте прагове, debounce, периоди и условия за сработване.

Нататък

- лед това мини към следва Źата глава, '%ебаунс на бутон', за да надгради < теку Źата идея в естествената “ следва Źа стъпка.

Глава 16. Дебаунс на бутон

Пример с polling:

```
if current_value != last_stable_value: # Проверяваме дали е настъпила промяна спрямо п
оследното стабилно състояние.
    now_ms = time.ticks_ms() # Вземаме текущото време само при промяна.
    if time.ticks_diff(now_ms, last_change_ms) >= 40: # Приемаме промяната за валидна
само ако е минало поне 40 ms.
        last_change_ms = now_ms # Обновяваме момента на последната приета промяна.
        last_stable_value = current_value # Приемаме новата стойност за стабилна.
```

Пример с `uasyncio`:

```
if stable_value == 0 and stable_count == 4: # Ако видим четири поредни проби с 0, прие-  
маме валидно натискане.  
    async_presses += 1 # Увеличаваме броя на валидните асинхронни натискания.
```

Тук има два подхода:

- времеви прозорец в polling цикъл
- последователни стабилни проби в async задача

‡ айл:

- `examples/23_timing/03_button_debounce.py`

Резюме

- ...лавата развива темата ‘%ебаунс на бутон’ около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Ёоя времева или събитийна идея е кл „ чова в ‘%ебаунс на бутон’?
2. Ёак бихте разграничили удобна демонстрация от наде f дно реално поведение?
3. Ёое в тази глава би се повредило най-лесно при твърде блокира Ź код?

Упражнения

1. Променете интервал, период или `timeout` и проследете разликата в поведението.
2. %обавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. • равнете поведението на два различни подхода от главата върху една и съ Źа задача.

Ако не работи

- • имптом: поведението е нестабилно във времето. Проверка: търсете дълги блокира Źи участъци и неправилни единици за време.
- • имптом: получавате твърде много или твърде малко събития. Проверка: ви f те прагове, `debounce`, периоди и условия за сработване.

Нататък

- лед това мини към следва Źата глава, ‘RTC’, за да надгради < теку Źата идея в естествената “ следва Źа стъпка.

Глава 17. RTC

Пример:

```
from machine import RTC # Импортираме RTC, за да работим с часовника в реално време.
```

```
rtc = RTC() # Създаваме обект за достъп до вградения RTC модул.  
print(rtc.info()) # Печатаме диагностичната информация за стартиране на RTC.  
print(rtc.calibration()) # Печатаме текущата калибрационна стойност.  
print(rtc.datetime()) # Четем текущата дата и час като осемелементен tuple.
```

Тази глава покрива:

- диагностика на RTC
- current datetime tuple
- безопасно задаване на време зад флаг

† айл:

- `examples/23_timing/06_rtc_basic.py`

Резюме

- ...главата развива темата 'RTC' около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Ёоя времева или събитийна идея е кл „чова в 'RTC'?
2. Ёак бихте разграничили удобна демонстрация от наде fдно реално поведение?
3. Ёое в тази глава би се повредило най-лесно при твърде блокира Ź код?

Упражнения

1. Променете интервал, период или timeout и проследете разликата в поведението.
2. %обавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. • равнете поведението на два различни подхода от главата върху една и съ Źа задача.

Ако не работи

- • имптом: поведението е нестабилно във времето. Проверка: търсете дълги блокира Źи участъци и неправилни единици за време.
- • имптом: получавате твърде много или твърде малко събития. Проверка: ви fте прагове, debounce, периоди и условия за сработване.

Нататък

- лед това мини към следва Źата глава, 'I2C master и runtime избор на пинове', за да надгради < теку Źата идея в естествената “ следва Źа стъпка.

Част V: Сериен интерфейс

Глава 18. I2C master и runtime избор на пинове

Създаване на пример:

```
from machine import I2C # Импортираме I2C за работа като master върху наличния хардуер
ен контролер.
```

```
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на 100 kHz върху бордовите пинове P100 и
P101.
```

```
print(i2c.scan()) # Сканираме шината за устройства с 7-битов адрес.
```

Runtime пинове:

```
from machine import I2C, Pin # Импортираме I2C и Pin, за да покажем runtime избиране на
SCL и SDA.
```

```
i2c = I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2C(1) на 400
kHz с изрично зададени пинове.
```

Тази глава е важна и заради конфликтите:

- I2C(0) ползва P400/P401
- P400 е и SW1

Файлове:

- examples/24_i2c/01_i2c_master_basic.py
- examples/24_i2c/02_i2c_runtime_pins.py

Резюме

- ...лавата въвежда интерфейса 'I2C master и runtime избор на пинове' като реален комуникационен канал с външен свят.
- Фокусът е върху линиите, ролите на участниците и минималния практически тест.
- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Кога е основната роля на интерфейса в 'I2C master и runtime избор на пинове'?
2. Кой параметър или сигнал е най-критичен за първоначален успех на примера?
3. Как бихте валидирали комуникацията преди да включите реално външно устройство?

Упражнения

1. Направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. Напишете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- имптом: няма отговор от < ината. Проверка: първо сверете пиновете, адреса/ролята и нуфните pull-up или control сигнали.
- имптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

• лед това мини към следващата глава, 'SoftI2C и I2CTarget', за да надгради < текущата идея в естествената " следваща стъпка.

Глава 19. SoftI2C и I2CTarget

SoftI2C:

```
from machine import SoftI2C, Pin # Импортираме SoftI2C и Pin за софтуерна I2C шина върху GPIO.
```

```
soft_i2c = SoftI2C(scl=Pin("P105", Pin.OPEN_DRAIN), sda=Pin("P104", Pin.OPEN_DRAIN), freq=100000) # Създаваме SoftI2C на 100 kHz.
```

I2CTarget:

```
from machine import I2CTarget, Pin # Импортираме I2CTarget и Pin за работа в target и slave режим.
```

```
memory_buffer = bytearray(range(16)) # Подготвяме малък memory-backed буфер, който target-ът ще обслужва.
```

```
target = I2CTarget(1, addr=0x42, mem=memory_buffer, mem_addrsize=8, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2CTarget(1) върху стандартните пинове на канал 1.
```

Тук се вижда:

- софтуерна I2C < ина върху GPIO
- целево устройство с memory-backed буфер

файлове:

- examples/24_i2c/03_softi2c_basic.py
- examples/24_i2c/04_i2ctarget_memory.py

Резюме

- ...лавата въвежда интерфейса 'SoftI2C и I2CTarget' като реален комуникационен канал с вън < ен свят.
- Фокусът е върху линиите, ролите на участниците и минималния практически тест.
- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Ёоя е основната роля на интерфейса в 'SoftI2C и I2CTarget'?
2. Ёой параметър или сигнал е най-критичен за първоначален успех на примера?

3. Как бихте валидирали комуникацията преди да вкл., чите реално външно устройство?

Упражнения

1. • направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. , пижете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- импотом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нуфните pull-up или control сигнали.
- импотом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

•лед това мини към следващата глава, 'Hardware SPI и SoftSPI', за да надградиш текущата идея в естествената "следваща стъпка.

Глава 20. Hardware SPI и SoftSPI

Hardware SPI:

```
from machine import SPI, Pin # Импортираме SPI и Pin за достъп до хардуерния SPI канал
```

```
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P100"), cs=Pin("P103")) # Създаваме SPI обект върху валидните board пинове.
```

SoftSPI:

```
from machine import SoftSPI, Pin # Импортираме SoftSPI и Pin за софтуерен SPI върху GPIO.
```

```
soft_spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=Pin("P105"), mosi=Pin("P104"), miso=Pin("P106")) # Създаваме софтуерен SPI обект върху три GPIO пина.
```

- тази книга SPI остава 1 канал.

файлове:

- examples/25_spi/01_spi_basic.py
- examples/25_spi/02_softspi_basic.py

Резюме

- ...лавата въвежда интерфейса 'Hardware SPI и SoftSPI' като реален комуникационен канал с външен свят.
- фокусът е върху линиите, ролите на участниците и минималния практически тест.

- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Каква е основната роля на интерфейса в 'Hardware SPI и SoftSPI'?
2. Кой параметър или сигнал е най-критичен за първоначален успех на примера?
3. Как бихте валидирали комуникацията преди да вкл., чите реално външно устройство?

Упражнения

1. Направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. Напишете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- симптом: няма отговор от външната. Проверка: първо сверете пиновете, адреса/ролята и нуфните pull-up или control сигнали.
- симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

- след това мини към следващата глава, 'TouchPad: базово четене', за да надгради текущата идея в естествената "следваща стъпка."

Част VI: TouchPad, памет и системни функции

Глава 21. TouchPad: базово четене

Пример:

```
from machine import Pin, TouchPad # Импортираме Pin и TouchPad за работа с CTSU вход.

touch = TouchPad(Pin("P205")) # Създаваме TouchPad върху валиден touch пин P205.
touch.config(500) # Задаваме праг 500 за метода value().
print(touch.read()) # Четем необработената CTSU стойност.
print(touch.value()) # Преобразуваме стойността към 0 или 1 според прага.
print(touch.read_value()) # Взимаме едновременно count, value и последна FSP грешка.
```

Тук се виждат:

- сурова стойност
- логическа стойност по праг
- комбинирано четене

† айл:

- `examples/26_touchpad/01_touchpad_basic.py`

Резюме

- ...лавата развива темата 'TouchPad: базово четене' около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и режими на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. коя е най-важната особеност или ограничение в 'TouchPad: базово четене'?
2. кой ресурс е *volatile* и кой е *persistent* в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един клон, чов праг, буфер или режим и сравнете поведението преди и след това.
3. • върнете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- импотом: поведението е чувствително или кумно. Проверка: търсете грешни прагове, неподходящи режим или неправилни очаквания към хардуера/порта.
- импотом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеш и порт-специфични особености.

Нататък

- след това мини към следващата глава, 'TouchPad: диагностика, cached API и асинхронно следене', за да надгради текущата идея в естествената "следваща стъпка.

Глава 22. TouchPad: диагностика, cached API и асинхронно следене

Диагностичен пример:

```
TouchPad.sample_rate(20) # Стартираме глобалния cooperative sampler на 20 пълни scan-a
                          # в секунда.
touch.start() # Стартираме non-blocking scan, ако не е в ход такъв.
TouchPad.service() # Придвижваме вътрешния sampler една стъпка от VM контекст.
print(touch.ready(), touch.read_cached(), touch.value_cached()) # Показваме cached съ-
стоянието.
print(touch.diagnose(8)) # Печатаме резултат от кратък диагностичен цикъл.
```

^синхронен пример:

```

TouchPad.sample_rate(50) # Стартираме sampler на 50 scan-a в секунда.

async def touch_task(): # Асинхронна задача за следене на TouchPad.
    while True: # Безкрайно следене.
        if tp.ready(): # Проверяваме дали пробата е готова.
            print(tp.read_cached(), tp.value_cached(), tp.age_ms()) # Печатаме cached
стойностите.
            await asyncio.sleep_ms(20) # Сканираме на 20 ms.

```

Тази глава е *важна*, защото показва не само 'натиснато/ненатиснато', а и сервизно поведение на CTSU слоя.

џайлове:

- `examples/26_touchpad/02_touchpad_diagnostics.py`
- `examples/04_touchpad_async.py`

Резюме

- ...лавата развива темата 'TouchPad: диагностика, cached API и асинхронно следене' около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; *важно* е да разбирате ограниченията и *режима* на работа.
- Това е зоната, в която практическата дисциплина става толкова *важна*, колкото и самият пример.

Самопроверка

1. Ёоя е най-важната особеност или ограничение в 'TouchPad: диагностика, cached API и асинхронно следене'?
2. Ёой ресурс е *volatile* и кой е *persistent* в този тип пример?
3. Ёак бихте измерили или набл., давали дали примерът се държи стабилно?

Упражнения

1. %обавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един кл., чов праг, буфер или *режим* и сравнете поведението преди и след това.
3. • върнете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- импотом: поведението е чувствително или < умно. Проверка: търсете грешни прагове, неподходящ *режим* или неправилни очаквания към хардуера/порта.
- импотом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, *кеш* и порт-специфични особености.

Нататък

- Вед това мини към следващата глава, 'renesas.Flash, /flash и dataflash', за да надгради текущата идея в естествената "следваща стъпка.

Глава 23. renesas.Flash, /flash и dataflash

Пример с renesas.Flash:

```
import os # Импортираме os, за да покажем съдържанието на файловата система /flash.
import renesas # Импортираме renesas модула, който излага Flash block device.

flash = renesas.Flash() # Създаваме block device обект върху вътрешната flash памет.
sector = bytearray(16) # Подготвяме малък буфер за четене на първите байтове от блок 0
.
flash.readblocks(0, sector) # Четем първите 16 байта от блок 0 в буфера.
print(os.listdir("/flash")) # Показваме кои файлове има във вътрешната файлова система
.
```

Пример с dataflash:

```
import dataflash # Импортираме dataflash модула за достъп до отделната Data Flash област.

print(dataflash.size()) # Печатаме общия размер на Data Flash областта.
print(dataflash.block_size()) # Печатаме размера на erase блока.
print(dataflash.write_size()) # Печатаме минималния размер за запис.
print(dataflash.read(0, 16)) # Четем първите 16 байта от Data Flash.
```

Пример за устойчиво състояние в dataflash и видима индикация с LED1:

```
from machine import Pin # Импортираме Pin за управление на LED1.
import dataflash # Импортираме dataflash за устойчиво съхранение на състояние.

led = Pin("LED1", Pin.OUT, value=1) # Използваме board alias-а LED1, започваме изгасен
.
state = dataflash.read(0, 1)[0] # Връщаме стойността на първия байт.

if state == 0x00: # Ако предишният старт е записал 0x00, сега правим erase към 0xFF.
    dataflash.erase_block(0) # Изтриваме блок 0, всички байтове стават 0xFF.
    blinks_per_sec = 2 # Бавно мигане означава erase.
else: # Иначе приемаме, че байтът е 0xFF и записваме 0x00.
    dataflash.write(0, bytes([0x00])) # Записваме 0x00 в първия байт.
    blinks_per_sec = 10 # Бързо мигане означава write.
```

Този пример показва как един байт в dataflash може да пази състояние между два старта на програмата.

За VK_RA4M2 LED индикацията е върху LED1, защото провереният board LED е P204 и е active-low.

При всяко стартиране се редуват две състояния:

- 0xFF след erase_block(0)
- 0x00 след write(0, bytes([0x00]))
- коростта на мигане показва кой преход е станал на текущия boot.

Тук се разграничават:

- файлова система
- block device
- отделен data flash регион

Проверени размери за текущата конфигурация на VK_RA4M2:

- MCU вариантът на тази платка е с общо 384 KB code flash и 128 KB RAM
- в текущия linker layout 290 KB са отделени за firmware/application region, а 94 KB са отделени като FLASH_FS за /flash
- отделно има 8 KB dataflash
- /flash е върху вътрешния flash block device, монтиран като FAT; block size е 512 bytes
- реално свободното място за файлове е малко по-малко от 94 KB, защото част отива за самата файлова система

Файлове:

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

- ...лавата развива темата 'renesas.Flash, /flash и dataflash' около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и рефима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Каква е най-важната особеност или ограничение в 'renesas.Flash, /flash и dataflash'?
2. Какъв ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали, дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един клон, човешки праг, буфер или рефим и сравнете поведението преди и след това.
3. • върнете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- импотом: поведението е чувствително или < умно. Проверка: търсете гре < ни прагове, неподходящ ref им или неправилни очаквания към хардуера/порта.
- импотом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, ке < иране и порт-специфични особености.

Нататък

- след това мини към следващата глава, '†правление на паметта в MicroPython', за да надгради < текущата идея в естествената " следващата стъпка.

Глава 24. Управление на паметта в MicroPython

%иагностичен пример:

```
import gc # Импортираме gc за контрол и измерване на garbage-collected heap-a.
import micropython # Импортираме micropython за mem_info и други runtime helper-и.

gc.collect() # Пускаме GC в предвидим момент, за да започнем от по-чисто състояние.
print(gc.mem_free(), gc.mem_alloc()) # Печатаме свободния и заетия heap.
micropython.mem_info() # Печатаме обобщена информация за heap-a.
```

Щуфeрен пример:

```
buffer = bytearray(32) # Създаваме един буфер, който ще използваме.
window = memoryview(buffer)[8:12] # Вземаме прозорец към част от буфера без копиране.
window[0] = 123 # Променяме първия байт през memoryview.
print(list(buffer[8:12])) # Показваме, че оригиналният буфер е променен на място.
```

Тук 'управление на паметта' не означава ръчно местене на адреси както в C, а по-скоро:

- да разбираме кои обекти заемат място в heap-a на MicroPython
- да различаваме RAM heap от stack, firmware и постоянна flash памет
- да ограничаваме излизателните алокации в чувствителни по време участъци
- да използваме буфери вместо да създаваме нови обекти във всеки цикъл

Потвърдени факти за текущия VK_RA4M2 порт:

- garbage-collected heap-ът се подава на MicroPython чрез linker символите _heap_start и _heap_end
- linker-ът заделя отделен фиксиран stack регион от 8 KB
- /flash и dataflash са постоянна памет за файлове и данни, а не RAM heap за Python обекти

Практични API-та, които реално са налични в този порт:

- gc.collect() за принудително стартиране на garbage collector в удобен момент

- `gc.mem_free()` и `gc.mem_alloc()` за ориентир колко heap е свободен и колко е зает
- `gc.threshold()` за по-напреднал контрол кога да се задейства GC
- `micropython.mem_info()` и `micropython.mem_info(1)` за по-подробна картина на heap-а
- `micropython.alloc_emergency_exception_buf()` за изкл. „чително ваfни exception-и в interrupt/critical контекст
- `micropython.heap_lock()` и `micropython.heap_unlock()` за напреднали случаи, когато временно не трябва да се допуска алокация

Практически правила, които най-често дават реален ефект:

- правете bytearray веднъf и после го пълнете отново, вместо да създавате нов буфер всеки път
- използвайте memoryview, когато искате прозорец към част от буфера без копиране
- ако имате временни големи обекти, освободf давайки ги и извиквайте `gc.collect()` в предвидим момент, а не насред критична операция
- при interrupt-и, callbacks и latency-sensitive код избягвайте нови алокации; подгответе буферите предварително
- не смесвайте `machine.mem8/mem16/mem32` с управлението на Python heap-а; това са различни нива на достъп до паметта

Тази тема се свързва естествено с вече наличните примери:

- `07_pointers_equivalents.py` показва shared reference и memoryview
- `04_i2ctarget_memory.py` показва предварително заделен mutable буфер, подаден към периферен API
- `01_flash_blockdev.py` и `02_dataflash_basic.py` показват, че persistent storage не е съfто като heap RAM

† айлове:

- `examples/28_machine_misc/04_memory_management.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/27_storage/01_flash_blockdev.py`
- `examples/27_storage/02_dataflash_basic.py`

Резюме

- ...лавата развива темата ‘†правление на паметта в MicroPython’ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; ваfно е да разбирате ограниченията и реfима на работа.
- Това е зоната, в която практическата дисциплина става толкова ваfна, колкото и самият пример.

Самопроверка

1. Ёоя е най-важната особеност или ограничение в 'управление на паметта в MicroPython'?
2. Ёой ресурс е volatile и кой е persistent в този тип пример?
3. Ёак бихте измерили или набл., давали дали примерът се държи и стабилно?

Упражнения

1. %обавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един кл., чов праг, буфер или рефим и сравнете поведението преди и след това.
3. • върнете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- импотом: поведението е чувствително или < умно. Проверка: търсете грешни прагове, неподходящ рефим или неправилни очаквания към хардуера/порта.
- импотом: резултатът не съпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеш иране и порт-специфични особености.

Нататък

• след това мини към следващата глава, 'machine модулът и порт-специфично поведение', за да надградите текущата идея в естествената "следваща стъпка.

Глава 25. machine модулът и порт-специфично поведение

• информативен пример:

```
import machine # Импортираме machine, за да покажем системните функции на порта.

uid = machine.unique_id() # Четем уникалния 128-битов идентификатор на MCU.
uid_hex = "".join("{:02x}".format(byte_value) for byte_value in uid) # Преобразуваме байтовете до четим hex низ.

print("machine.freq() =", machine.freq()) # Показваме текущата честота на ядрото.
print("machine.unique_id() =", uid_hex) # Печатаме уникалния идентификатор в hex вид.
print("machine.reset_cause() =", machine.reset_cause()) # Печатаме причината за последния reset.

machine.info() # Печатаме подробна системна информация от самия порт.
```

• ИСКО НИВО:

```
irq_state = machine.disable_irq() # Временно забраняваме прекъсванията и пазим предишното състояние.

machine.enable_irq(irq_state) # Възстановяваме прекъсванията веднага след демонстрацията.

print(hasattr(machine, "mem32")) # Показваме дали портът излага 32-битов memory access.
```

Sleep режими:

```

DO_LIGHTSLEEP = False # Оставяме lightsleep изключен по подразбиране.
DO_DEEPSLEEP = False # Оставяме deepsleep изключен по подразбиране.

print("Извикваме кратък machine.lightsleep(20).") # Подготвяме потребителя за кратък sleep.
machine.lightsleep(20) # Извикваме кратък lightsleep за 20 ms.

if DO_LIGHTSLEEP: # Само ако е включено, пробваме по-дълъг lightsleep.
    machine.lightsleep(100) # Извикваме lightsleep за 100 ms.

```

Тази глава е мястото за:

- machine.info
- machine.freq
- machine.unique_id
- machine.reset_cause
- disable_irq и enable_irq
- mem8, mem16, mem32
- предпазен machine.bootloader
- sleep, lightsleep, deepsleep

Практическа бележка за честоти и ток:

- machine.freq() е полезен за бърза проверка на runtime честотата; текущата board конфигурация е 100 MHz системен clock и 50 MHz периферни clock-и за много блокове
- токовете числа в hardware manual-а са за самия MCU при конкретни условия, а не за цялата платка през USB
- типични стойности от manual-а са около 15.4 mA в normal mode с всички peripheral clocks enabled, 6.1 mA при disabled peripheral clocks и 4.4 mA в sleep mode
- за ниска консумация manual-ът дава ориентири от 0.7 uA за software standby и приблизително 5 ... 16 uA за deep software standby според retained функциите
- по време на 12-bit A/D conversion analog current е типично около 0.8 mA, което е отделна аналогова консумация и не бива да се бърка с общата консумация на платката

џайлове:

- examples/28_machine_misc/01_machine_info_and_id.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

- ...лавата развива темата 'machine модулът и порт-специфично поведение' около по-чувствителни, по-системни или порт-специфични възможности.

- Тук вече не е достатъчно само да извикате API-то; *важно* е да разбирате ограниченията и *режима* на работа.
- Това е зоната, в която практическата дисциплина става толкова *важна*, колкото и самият пример.

Самопроверка

1. *Коя* е най-*важната* особеност или ограничение в 'machine модулът и порт-специфично поведение'?
2. *Кой* ресурс е *volatile* и *кой* е *persistent* в този тип пример?
3. *Как* бихте измерили или *набл.*, *давали* дали примерът се *държи* и стабилно?

Упражнения

1. *Обавете* диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един *кл.*, *чов* праг, буфер или *режим* и сравнете поведението преди и след това.
3. *Върнете* темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е *приложимо*.

Ако не работи

- *импотом*: поведението е чувствително или *к* умно. Проверка: търсете *грешни* прагове, неподходящи *режим* или неправилни очаквания към хардуера/порта.
- *импотом*: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, *к* иране и порт-специфични особености.

Нататък

- *лед* това мини към следващата глава, 'динамична 7-сегментна индикация и матрично сканиране', за да надгради *текущата* идея в естествената "следваща стъпка".

Част VII: Индикация и входни матрици

Глава 26. Динамична 7-сегментна индикация и матрично сканиране

Пример:

```
from machine import Pin # Импортираме Pin, за да покажем базовото сканиране на външен
дисплей и бутони.

import time # Импортираме time за краткото динамично опресняване.

segment_pins = [Pin(name, Pin.OUT) for name in ("P105", "P104", "P113", "P114", "P115",
"P304", "P303")] # Това са седемте сегмента а..g.

digit_pins = [Pin(name, Pin.OUT) for name in ("P608", "P409")] # Това са двата общи из
вода за две цифри.

patterns = {"1": (0, 1, 1, 0, 0, 0, 0), "2": (1, 1, 0, 1, 1, 0, 1)} # Това са сегментн
ите шаблони за две примерни цифри.

for refresh_step in range(10): # Повтаряме кратък опреснителен цикъл десет пъти.
```

```

    for digit_index, symbol in enumerate(("1", "2")): # Показваме символ 1 на първата
цифра и 2 на втората цифра.

        for digit_pin in digit_pins: # Първо изключваме всички цифри, за да няма ghost
ing.

            digit_pin.off() # Изключваме текущата цифра.

            for segment_pin, level in zip(segment_pins, patterns[symbol]): # Зареждаме нуж
ния сегментен шаблон за текущия символ.

                segment_pin.value(level) # Настройваме сегмента според шаблона.

            digit_pins[digit_index].on() # Включваме само текущата цифра за кратко време.

            time.sleep_ms(5) # Държим цифрата включена кратко, за да се получи динамична и
ндикация.

```

Клавиатурна матрица:

```

row_pins = [Pin(name, Pin.OUT) for name in ("P408", "P600")] # Това са редовете на при
мерна 2x2 клавиатурна матрица.

col_pins = [Pin(name, Pin.IN, Pin.PULL_UP) for name in ("P301", "P302")] # Това са кол
оните на примерната 2x2 клавиатурна матрица.

pressed_keys = [] # Подготвяме списък за натиснати клавиши от матрицата.

for row_index, row_pin in enumerate(row_pins): # Сканираме всеки ред поотделно.

    for reset_row in row_pins: # Първо изключваме всички редове.

        reset_row.off() # Изключваме реда.

        row_pin.on() # Активираме текущия ред.

        for col_index, col_pin in enumerate(col_pins): # Проверяваме всяка колона в активн
ия ред.

            if col_pin.value() == 0: # Ако колоната падне на 0, приемаме че бутонът е нати
снат.

                pressed_keys.append((row_index, col_index)) # Записваме координатите на на
тиснатия бутон.

```

Този пример изисква външен хардуер, но е много важен за свързване на GPIO, време и логика.

Файл:

- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`

Резюме

- Програмата комбинира GPIO, време и външен хардуер в една по-сложна, но много показателна демонстрация.
- , сновната идея е, че мултиплексирането и матричното сканиране разчитат на правилен ред и темпо на опресняване.
- Това е добър пример как софтуерното време пряко влияе на видимия хардуерен резултат.

Самопроверка

1. Защо динамичната индикация не работи добре при твърде бавно опресняване?
2. Каква е идеята зад матричното сканиране на клавиши?
3. Кой проблем ще се появи първо, ако редът на управление на сегменти и цифри е грешен?

Упражнения

1. Променете честотата на опресняване и наблюдавайте, давайки кога започва видимо мигане.
2. Набелязвайте проста таблица за превод от координати на матрицата към символи.
3. Използвайте LED1 като диагностичен индикатор при намерен натиснат клавиш.

Ако не работи

- симптом: дисплеят е слаб или премига. Проверка: периодът на опресняване вероятно е твърде дълъг или кодът блокира междувидовите цикли.
- симптом: разчитането на клавиши е грешно. Проверка: прегледайте съответствието между редове, колони и реалното окабеляване.

Нататък

- Веднага мини към следващата глава, 'Минимална FSM и таблична FSM', за да надградиш текущата идея в естествената "следваща стъпка".

Част VIII: FSM като начин на мислене

Глава 27. Минимална FSM и таблична FSM

Минимална FSM:

```
STATE_OFF = "OFF" # Това е първото състояние на лампата.
```

```
STATE_ON = "ON" # Това е второто състояние на лампата.
```

```
EVENT_BUTTON = "BUTTON" # Това е събитието от бутона.
```

```
state = STATE_OFF # Започваме с изключена лампа.
```

```
def handle_event(current_state, event_name): # Дефинираме преходната функция на машината на състояния.
```

```
    if current_state == STATE_OFF and event_name == EVENT_BUTTON: # Ако сме в OFF и бутонът е натиснат, минаваме в ON.
```

```
        return STATE_ON # Връщаме новото състояние ON.
```

```
    if current_state == STATE_ON and event_name == EVENT_BUTTON: # Ако сме в ON и бутонът е натиснат, минаваме в OFF.
```

```
        return STATE_OFF # Връщаме новото състояние OFF.
```

```
    return current_state # Ако няма преход, оставаме в същото състояние.
```

Таблична FSM:

```
state = "OFF" # Започваме в състояние OFF.
```

```
transition_table = {("OFF", "BUTTON"): "ON", ("ON", "BUTTON"): "OFF"} # В таблицата оп  
исваме преходите като двойка от state и event.
```

```
event_sequence = ["BUTTON", "BUTTON", "BUTTON", "BUTTON"] # Симулираме поредица от съб  
ития.
```

...рафично идеята мо~~е~~ е да се мисли така:

```
OFF --BUTTON--> ON
```

```
ON  --BUTTON--> OFF
```

†айлове:

- `examples/30_fsm/01_fsm_lamp_button.py`
- `examples/30_fsm/02_fsm_table.py`

Резюме

- ...лавата развива системното мислене чрез темата 'Минимална FSM и таблична FSM'.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в 'Минимална FSM и таблична FSM'?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Ёое ре < ение тук помага най-много за бъде Ъо раз < иряване на системата?

Упражнения

1. %обавете о Ъе едно състояние, събитие, guard или правило според темата на главата.
2. , тпечатвайте теку Ъото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с о Ъе един реален вход.

Ако не работи

- • имптом: системата влиза в неочаквано състояние. Проверка: търсете липсва Ъ преход, гре < ен приоритет или слабо дефиниран event/condition.
- • имптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помо Ъни функции или таблици.

Нататък

- лед това мини към следва Ъата глава, 'FSM с timeout', за да надгради < теку Ъата идея в естествената “ следва Ъа стъпка.

Глава 28. FSM с timeout

Пример:

```
state = "OFF" # Започваме с изключена лампа.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "TIMEOUT"): "OFF", ("ON", "BUTTON")
: "OFF"} # Описваме преходите с timeout.

event_sequence = ["BUTTON", "TIMEOUT", "BUTTON", "BUTTON"] # Симулираме бутон и timeout събития.
```

Тази глава е преходът от 'лампа + бутон' към 'лампа + бутон + време'.

† айл:

- `examples/30_fsm/03_fsm_timeout.py`

Резюме

- ...лавата развива системното мислене чрез темата 'FSM с timeout'.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в 'FSM с timeout'?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Ёое ре < ение тук помага най-много за бъде Ѓо раз < иряване на системата?

Упражнения

1. Ёобавете о Ѓе едно състояние, събитие, guard или правило според темата на главата.
2. , тпечатвайте теку Ѓото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с о Ѓе един реален вход.

Ако не работи

- • имптом: системата влиза в неочаквано състояние. Проверка: търсете липсва Ѓ преход, гре < ен приоритет или слабо дефиниран event/condition.
- • имптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помо Ѓни функции или таблици.

Нататък

- лед това мини към следва Ѓата глава, 'Conditions vs Events', за да надгради < теку Ѓата идея в естествената " следва Ѓа стъпка.

Глава 29. Conditions vs Events

Пример:

```
light_level = 600 # Това е моментното ниво от светлинния сензор в симулацията.
```

```

dark_threshold = 800 # Това е прагът, под който приемаме че е тъмно.
event_name = "BUTTON" # Това е дискретното събитие от бутон.

condition_is_dark = light_level < dark_threshold # Това е condition, която може да е в
ярна дълго време.

if condition_is_dark and event_name == "BUTTON": # Комбиниране условие и събитие в общ
о решение.

    print("Лампата може да се включи, защото е тъмно и има бутонно събитие.") # Печата
ме положителния изход.

else: # Ако комбинацията не е изпълнена, не включваме лампата.

    print("Няма достатъчно условия за включване.") # Печатаме отрицателния изход.

```

Тук разграничението е:

- condition може да е вярно дълго време
- event е дискретен факт

џаил:

- `examples/30_fsm/04_conditions_vs_events.py`

Резюме

- ...главата развива системното мислене чрез темата 'Conditions vs Events'.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в 'Conditions vs Events'?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Ёое ре < ение тук помага най-много за бъде ъо раз < иряване на системата?

Упражнения

1. %обавете о ъе едно състояние, събитие, guard или правило според темата на главата.
2. , тпечатвайте теку ъото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с о ъе един реален вход.

Ако не работи

- • имптом: системата влиза в неочаквано състояние. Проверка: търсете липсва ъ преход, гре < ен приоритет или слабо дефиниран event/condition.
- • имптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помо ъни функции или таблици.

Нататък

- лед това мини към следващата глава, 'Guard функции', за да надгради < текущата идея в естествената " следваща стъпка.

Глава 30. Guard функции

Пример:

```
context = {"light": 500, "pir": True} # Подготвяме контекст със стойности от сензори.
state = "OFF" # Започваме в състояние OFF.
event_name = "MOTION" # Симулираме събитие за засечено движение.

def is_dark(data): # Това е първата guard функция.
    return data["light"] < 800 # Връщаме True само ако е достатъчно тъмно.

def has_motion(data): # Това е втората guard функция.
    return data["pir"] is True # Връщаме True само ако PIR сензорът вижда движение.

if state == "OFF" and event_name == "MOTION" and is_dark(context) and has_motion(context): # Комбинираме event, state и guard функции.
    state = "ON" # Минаваме в ON само ако всички проверки са успешни.
```

Тук преходът не зависи само от state и event, а и от допълнителни условия.

† айл:

- examples/30_fsm/05_guard_functions.py

Резюме

- ...лавата развива системното мислене чрез темата 'Guard функции'.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в 'Guard функции'?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Ёое ре < ение тук помага най-много за бъде Ъо раз < иряване на системата?

Упражнения

1. %обавете оЪе едно състояние, събитие, guard или правило според темата на главата.
2. , тпечатвайте текуЪото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с оЪе един реален вход.

Ако не работи

- имптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, гре < ен приоритет или слабо дефиниран event/condition.
- имптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

• лед това мини към следващата глава, 'Приоритети и event queue', за да надгради < текущата идея в естествената " следваща стъпка.

Глава 31. Приоритети и event queue

Приоритети:

```
pending_events = ["BUTTON", "ERROR", "TIMEOUT"] # Симулираме едновременно чакащи събития.

priority_order = {"ERROR": 0, "BUTTON": 1, "TIMEOUT": 2} # По-малкото число означава по-висок приоритет.

pending_events.sort(key=lambda event_name: priority_order[event_name]) # Подреждаме събитията по приоритет.
```

Event queue:

```
event_queue = [] # Създаваме проста event queue като обикновен Python list.

event_queue.append(("BUTTON", {"source": "SW1"})) # Добавяме събитие от бутон.
event_queue.append(("PIR", {"source": "pir_sensor"})) # Добавяме събитие от PIR сензор.
event_queue.append(("TIMEOUT", {"ms": 3000})) # Добавяме timeout събитие.

while event_queue: # Обработваме опашката, докато има чакащи събития.
    event_name, payload = event_queue.pop(0) # Взимаме най-старото събитие от началото на опашката.
    print("Обработваме", event_name, "с payload", payload) # Печатаме кое събитие се обработва сега.
```

Това е мостът между:

- прекъсвания
- таймери
- сензори
- FSM логика

файлове:

- examples/30_fsm/06_priorities.py
- examples/30_fsm/07_event_queue.py

Резюме

- Главата развива системното мислене чрез темата 'Приоритети и event queue'.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в 'Приоритети и event queue'?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Ёое ре < ение тук помага най-много за бъде Ъо раз < иряване на системата?

Упражнения

1. %обавете о Ъе едно състояние, събитие, guard или правило според темата на главата.
2. , тпечатвайте теку Ъото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с о Ъе един реален вход.

Ако не работи

- имптом: системата влиза в неочаквано състояние. Проверка: търсете липсва Ъ преход, гре < ен приоритет или слабо дефиниран event/condition.
- имптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помо Ъни функции или таблици.

Нататък

- лед това мини към следва Ъата глава, 'Tickless timers и sleep идея', за да надгради < теку Ъата идея в естествената " следва Ъа стъпка.

Глава 32. Tickless timers и sleep идея

Пример:

```
import machine # Импортираме machine за кратък lightsleep между събитията.
import time   # Импортираме time за работа с ticks_ms и ticks_diff.

tasks = [("lamp_timeout", 120), ("fan_timeout", 450), ("display_refresh", 40)] # Подготвяме три задачи с различни крайни срокове в милисекунди.
start_ms = time.ticks_ms() # Запомняме началния момент.
next_deadline_ms = min(deadline for _, deadline in tasks) # Избираме най-близкия срок като tickless цел.
sleep_ms = max(0, next_deadline_ms - time.ticks_diff(time.ticks_ms(), start_ms)) # Изчисляваме колко време може да се спи до следващото събитие.
machine.lightsleep(sleep_ms) # Спим точно до най-близкото планирано събитие с lightsleep.
elapsed_ms = time.ticks_diff(time.ticks_ms(), start_ms) # Изчисляваме изминалото време след съня.
```

```
ready_tasks = [task_name for task_name, deadline in tasks if deadline <= elapsed_ms] #  
Избираме всички задачи, които вече са изтекли.
```

Тази глава не разчита на агресивни *sleep* рефими, а показва начина на мислене:

- намираме най-близкия срок
- спим точно дотогава
- обработваме всички изтекли задачи

†айлове:

- `examples/30_fsm/08_tickless_sleep.py`
- `examples/28_machine_misc/03_sleep_modes.py`

Резюме

- ...лавата развива системното мислене чрез темата ‘Tickless timers и sleep идея’.
- , сновният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и раз < иряем.

Самопроверка

1. Ёоя е кл „ човата моделна идея в ‘Tickless timers и sleep идея’?
2. Ёак тази глава прави поведението по-структурирано от чиста поредица от `if` проверки?
3. Ёое ре < ение тук помага най-много за бъде Ъо раз < иряване на системата?

Упражнения

1. %обавете о Ъе едно състояние, събитие, `guard` или правило според темата на главата.
2. , тпечатвайте теку Ъото състояние и последното обработено събитие за по-лесно проследяване.
3. , пи < ете с кратка диаграма или таблица как бихте раз < ирили модела с о Ъе един реален вход.

Ако не работи

- • имптом: системата влиза в неочаквано състояние. Проверка: търсете липсва Ъ преход, гре < ен приоритет или слабо дефиниран `event/condition`.
- • имптом: моделът става нечетим. Проверка: върнете се към ясни имена на `states`, `events` и отделни помо Ъни функции или таблици.

Нататък

- лед това мини към следва Ъата глава, ‘†правление на осветление и вентилация за баня’, за да надгради < теку Ъата идея в естествената “ следва Ъа стъпка.

Част IX: Реален пример

Глава 33. Управление на осветление и вентилация за баня

Пример:

```
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

led = Pin("LED1", Pin.OUT, value=1) # LED1 = P204, active-low: value=1 означава изгасен.

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, active-low: value=0 означава натиснат.

sensors = {"pir": True, "door_open": False, "light_level": 300, "humidity": 82} # Примерни входни данни от четири сензора.

outputs = {"lamp": False, "fan": False} # Два управлявани изхода за лампа и вентилатор.

button_pressed = (button.value() == 0) # SW1 е active-low: 0 означава натиснат.

if sensors["pir"] and sensors["light_level"] < 800: # Ако има движение и е тъмно, включваме лампата.
    outputs["lamp"] = True # Включваме лампата.

if button_pressed: # Ако бутонът е натиснат, също включваме лампата.
    outputs["lamp"] = True # Ръчно включване от потребителя.

if sensors["humidity"] > 75 or sensors["door_open"]: # Ако влажността е висока или вратата е отворена, включваме вентилатора.
    outputs["fan"] = True # Включваме вентилатора.
```

Това е умишлено учебна симулация, не готов продукт.

• о тя събира:

- движение
- врата
- осветеност
- влажност
- два управлявани изхода

файл:

- `examples/31_case_study/01_bathroom_control.py`

Резюме

- ...лавата прилага вече натрупаните идеи в по-реалистичен сценарий чрез 'управление на осветление и вентилация за баня'.
- Тук вече се мисли за цялостна система, а не за изолиран синтактичен или периферен пример.
- Практическата стойност е в това да видите как отделни техники започват да работят заедно.

Самопроверка

1. Ёоя част от '†правление на осветление и вентилация за баня' е реална и коя е уми < лено симулирана за учебни цели?
2. Ёои ре < ения в тази глава са добри за учебен пример, но оŽе не са завър < ен продукт?
3. Ёой преди < ен урок се използва най-видимо тук?

Упражнения

1. Променете един праг или правило и опи < ете как това влияе на цялата система.
2. %обавете кратък лог на ре < енията, които системата взема.
3. Разделете една част от логиката в помоŽна функция, за да направите кода по-четим.

Ако не работи

- имптом: логиката изгле f да твърде сло f на. Проверка: проследявайте отделно входовете, условията и изходите, вместо да гледате всичко наведнъ f.
- имптом: резултатът не отговаря на очакванията. Проверка: ви f те кои входове са реални и кои са само симулирани данни в примера.

Нататък

- лед това мини към следваŽата глава, '%екомпозиция на две FSM', за да надгради < текуŽата идея в естествената " следваŽа стъпка.

Глава 34. Декомпозиция на две FSM

Пример:

```
lamp_state = "OFF" # Това е текущото състояние на FSM за лампата.
fan_state = "OFF" # Това е текущото състояние на FSM за вентилатора.

event_sequence = [] # Започваме с празен списък.

if button.value() == 0: # Проверяваме дали SW1 е натиснат в момента.
    event_sequence.append("BUTTON") # Добавяме реално събитие от бутона.

event_sequence.extend(["HUMIDITY_HIGH", "TIMEOUT_LAMP", "TIMEOUT_FAN"]) # Добавяме симулирани събития.

for event_name in event_sequence: # Обработваме събитията последователно и за двете машини.
    if event_name == "BUTTON": # Бутонът управлява лампата.
        lamp_state = "ON" if lamp_state == "OFF" else "OFF" # Превключваме FSM на лампата.
    if event_name == "HUMIDITY_HIGH": # Високата влажност влияе на FSM на вентилатора.
        fan_state = "ON" # Включваме вентилатора.
```

Тук голямата идея е:

- една FSM да не поглъŽа цялата система
- отделните ма < ини да имат отделни отговорности

- комуникацията да става през събития

џайл:

- `examples/31_case_study/02_two_fsms.py`

Забележка: • двата case study примера вече използват реален LED1 (P204) и SW1 (P400) за вход и изход, освен симулираните сензори.

Резюме

- ...лавата прилага вече натрупаните идеи в по-реалистичен сценарий чрез ‘%екомпозиция на две FSM’.
- Тук вече се мисли за цялостна система, а не за изолиран синтактичен или периферен пример.
- Практическата стойност е в това да видите как отделни техники започват да работят заедно.

Самопроверка

1. Ёоя част от ‘%екомпозиция на две FSM’ е реална и коя е уми < лено симулирана за учебни цели?
2. Ёои ре < ения в тази глава са добри за учебен пример, но оџе не са завър < ен продукт?
3. Ёой преди < ен урок се използва най-видимо тук?

Упражнения

1. Променете един праг или правило и опи < ете как това влияе на цялата система.
2. %обавете кратък лог на ре < енията, които системата взема.
3. Разделете една част от логиката в помоџна функция, за да направите кода по-четим.

Ако не работи

- • имптом: логиката изгле & да твърде сло & на. Проверка: проследявайте отделно входовете, условията и изходите, вместо да гледате всичко наведнџ.
- • имптом: резултатът не отговаря на очакванията. Проверка: ви & те кои входове са реални и кои са само симулирани данни в примера.

Нататџк

- лед това мини към следваџата глава, ‘ѐазова UART комуникация’, за да надгради < текуџата идея в естествената “ следваџа стџпка.

Част X: UART комуникация

Глава 35. Базова UART комуникация

Пример:

```
from machine import UART # Импортираме UART класа за серийна комуникация.
import time # Импортираме time за кратко закъснение.

uart = UART(9, 115200) # Създаваме UART(9) на 115200 baud (TX=P602, RX=P601).
```

```

message = "Здравей от VK_RA4M2!\r\n" # Подготвяме съобщение за изпращане.
bytes_sent = uart.write(message) # Изпращаме съобщението по TX пина.

time.sleep_ms(100) # Кратко закъснение, за да пристигнат данни (ако има loopback).
available = uart.any() # Проверяваме дали има налични байтове за четене.
if available > 0: # Ако има данни за четене:
    received = uart.read(available) # Четем наличните байтове от RX буфера.
    print("Получено:", received) # Показваме получените данни.

```

• алични UART канали:

- UART(0) -> TX=P411, RX=P410
- UART(2) -> TX=P302, RX=P301
- UART(7) -> TX=P401, RX=P402, CTS=P403
- UART(9) -> TX=P602, RX=P601, CTS=P603

Тук се учат:

- създаване на UART инстанция
- изпращане и четене на данни
- loopback тест (TX свързан към RX)

‡ айл:

- examples/32_uart/01_uart_basic.py

Резюме

- ...лавата въве f да UART като най-простия сериен канал за текст и байтове.
- Показани са базово изпращане, наличие на данни и четене от буфера.
- Loopback тестът е естественият първи sanity check преди по-сложна комуникация.

Самопроверка

1. Каква е ролята на baud rate при UART комуникацията?
2. Защо loopback тестът е добър първи тест на конфигурацията?
3. Каква е практическата разлика ме f ду any() и read()?

Упражнения

1. • изпращайте прост текстов брояч веднъ f в секунда по UART.
2. ^ко имате loopback, отпечатвайте обратно полученото съобщение като echo тест.
3. %обавете функция за изпращане на текстов ред с край \r\n.

Ако не работи

- имптом: текстът е нечетим. Проверка: baud rate и настройките от двете страни трябва да съвпадат.

- импотом: няма върнати данни. Проверка: при loopback TX и RX трябва да са свързани по очаквания за теста начин.

Нататък

- лед това мини към следващата глава, ' , снови на asyncio', за да надгради < текущата идея в естествената " следваща стъпка.

Част XI: asyncio като учебна секция

Глава 36. Основи на asyncio

Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.

async def blink_task(period_ms): # Асинхронна задача за мигане с зададен период.
    while True: # Безкрайно мигане.
        led.value(LED_ON) # Светваме LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението за period_ms.
        led.value(LED_OFF) # Гасим LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението отново.
```

Тази глава покрива:

- create_task за стартиране на задачи
- asyncio.gather за паралелно изпълнение
- три задачи едновременно: мигане, бутон, отчет

файл:

- examples/33_asyncio/01_asyncio_basics.py

Резюме

- ...лавата систематизира asyncio чрез няколко кооперативни задачи.
- Показва се как LED, бутон и отчет могат да живеят като отделни роли в една програма.
- Това е реален практически модел за неблокиращи приложения върху платката.

Самопроверка

1. Какво прави create_task спрямо директно извикване на функция?

2. Защо `asyncio` е кооперативен, а не прекъсващ модел?
3. Ёга бихте избрали този подход пред много блокиращи цикли?

Упражнения

1. Добавете нова `async` задача, която периодично логва състоянието на приложението.
2. Променете периода на мигане според някакво събитие или брояч.
3. Препрочтайте една от задачите след `timeout` и проверете дали останалите продължават.

Ако не работи

- импотом: `event loop`-ът 'замръзва'. Проверка: някоя задача вероятно съдържа блокиращ код без `await`.
- импотом: дадена задача сякаш не съществува. Проверка: виждете дали реално е създадена и дали отстъпва управлението.

Нататък

- След това мини към следващата глава, 'asyncio Event синхронизация', за да надградиш текущата идея в естествената "следваща стъпка".

Глава 37. asyncio Event синхронизация

Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.
button_event = asyncio.Event() # Създаваме Event обект за синхронизация между задачи.

async def button_producer(): # Тази задача следи SW1 и сигнализира при натискане.
    last_value = 1 # Последна стойност на бутона (ненатиснат).
    current_value = button.value() # Четем SW1.
    if current_value == 0 and last_value == 1: # Нов натиск (falling edge).
        button_event.set() # Сигнализираме Event-а.

async def led_consumer(): # Тази задача чака Event и мига LED1.
    await button_event.wait() # Чакаме Event-а (блокираме кооперативно).
    button_event.clear() # Нулираме Event-а за следващо използване.
```

Тук се вижда:

- `asyncio.Event` за комуникация между задачи

- производител-консуматор модел
- кооперативно блокиране без busy-wait

‡ айл:

- `examples/33_asyncio/02_asyncio_event_flag.py`

Резюме

- ...главата показва `asyncio.Event` като чист механизъм за синхронизация ме f ду задачи.
- Моделът producer-consumer прави асинхронната комуникация по-ясна и по-наде f дна.
- Това е ва f на крачка от 'много задачи' към 'задачи, които си взаимодействат правилно'.

Самопроверка

1. Каква е ползата от `asyncio.Event` спрямо обикновен споделен флаг?
2. Защо консуматорът обикновено изчиства Event-а след обработка?
3. Как бихте разкрили примера, ако няколко задачи трябва да реагират на едно събитие?

Упражнения

1. Наблюдавайте брояч на Event събитията и го отпечатвайте периодично.
2. Създайте втори консуматор, който също чака същото събитие.
3. Наблюдавайте debounce или edge detection в producer задачата преди `set()`.

Ако не работи

- импотом: реакцията се случва само веднъж. Проверка: вижте дали Event-ът отново се задава след `clear()`.
- импотом: натискания се губят. Проверка: producer логиката и polling интервалът могат да са твърде груби.

Нататък

- Веднага тази глава прегледайте каталога на примерите и се върнете към темите, които искате да доразвиете на реален борд.

Каталог на всички примери

Този каталог е финалната проверка, че нито един пример не е пропуснат в книгата.

- `examples/01_blink_async.py` : асинхронно мигане на LED1
- `examples/02_two_tasks.py` : две едновременни `asyncio` задачи
- `examples/03_pwm_ab_same_channel.py` : два PWM изхода върху един GPT канал
- `examples/04_touchpad_async.py` : асинхронно следене на TouchPad с cached API
- `examples/20_language_basics/01_sample_program.py` : първа примерна програма
- `examples/20_language_basics/02_variables_and_constants.py` : променливи и константи
- `examples/20_language_basics/03_operators_conditions_loops.py` : оператори, условия и цикли

- `examples/20_language_basics/04_enums.py` : enum-подобни стойности с `const()`
- `examples/20_language_basics/05_arrays.py` : списъци, вложени таблици, `array`, `bytearray`
- `examples/20_language_basics/06_structures.py` : `dict`, `namedtuple`, вложени структури
- `examples/20_language_basics/07_pointers_equivalents.py` : `references`, `memoryview`, `machine.mem32`
- `examples/20_language_basics/08_functions.py` : функции, `callbacks` и таблици от функции
- `examples/21_digital_io/01_digital_output.py` : цифров изход с LED1
- `examples/21_digital_io/02_digital_input.py` : цифров вход с SW1
- `examples/22_analog/01_analog_input_adc.py` : ADC вътре < ни и вън < ни канали
- `examples/22_analog/02_pwm_output.py` : базов PWM изход
- `examples/23_timing/01_delays_and_timers.py` : блокиращо закъснение, хардуерен таймер, `uasyncio`
- `examples/23_timing/02_interrupts.py` : GPIO прекъсвания и защитено четене
- `examples/23_timing/03_button_debounce.py` : дебаунс с `polling` и с `uasyncio`
- `examples/23_timing/04_software_timer_minus1.py` : софтуерен `Timer(-1)`
- `examples/23_timing/05_timer_compare_hw_sw.py` : сравнение между `Timer(1)` и `Timer(-1)`
- `examples/23_timing/06_rtc_basic.py` : RTC основи
- `examples/23_timing/07_adc_timing_measure.py` : измерване на `ADC.read()` с `time.ticks_us()`
- `examples/23_timing/08_interrupt_latency_measure.py` : ориентировъчно измерване на GPIO interrupt latency
- `examples/24_i2c/01_i2c_master_basic.py` : базов I2C master
- `examples/24_i2c/02_i2c_runtime_pins.py` : I2C с runtime зададени пинове
- `examples/24_i2c/03_softi2c_basic.py` : `SoftI2C`
- `examples/24_i2c/04_i2ctarget_memory.py` : `I2CTarget` с `memory-backed` буфер
- `examples/25_spi/01_spi_basic.py` : hardware SPI
- `examples/25_spi/02_softspi_basic.py` : `SoftSPI`
- `examples/26_touchpad/01_touchpad_basic.py` : базов TouchPad
- `examples/26_touchpad/02_touchpad_diagnostics.py` : диагностика, `tuning` и `cached API`
- `examples/27_storage/01_flash_blockdev.py` : `renesas.Flash` и `/flash`
- `examples/27_storage/02_dataflash_basic.py` : `dataflash` модул
- `examples/27_storage/03_dataflash_state_led.py` : устойчиво състояние в `dataflash` и LED индикация
- `examples/28_machine_misc/01_machine_info_and_id.py` : `machine.info`, `freq`, `unique_id`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py` : `disable_irq`, `enable_irq`, `mem32`, `bootloader`
- `examples/28_machine_misc/03_sleep_modes.py` : `sleep`, `lightsleep`, `deepsleep`

- `examples/28_machine_misc/04_memory_management.py` : `gc`, `micropython.mem_info`, `bytearray`, `memoryview`
- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py` : динамична индикация и матрично сканиране
- `examples/30_fsm/01_fsm_lamp_button.py` : минимална FSM
- `examples/30_fsm/02_fsm_table.py` : таблична FSM
- `examples/30_fsm/03_fsm_timeout.py` : FSM с timeout
- `examples/30_fsm/04_conditions_vs_events.py` : `conditions` срещу `events`
- `examples/30_fsm/05_guard_functions.py` : `guard` функции
- `examples/30_fsm/06_priorities.py` : приоритети между събития
- `examples/30_fsm/07_event_queue.py` : `event queue`
- `examples/30_fsm/08_tickless_sleep.py` : `tickless` идея и `sleep`
- `examples/31_case_study/01_bathroom_control.py` : баня с няколко сензора и реален LED1/SW1
- `examples/31_case_study/02_two_fsms.py` : лампа и вентилатор като две FSM с реален LED1/SW1
- `examples/32_uart/01_uart_basic.py` : базова UART комуникация и `loopback` тест
- `examples/33_asyncio/01_asyncio_basics.py` : `asyncio` основи с три паралелни задачи
- `examples/33_asyncio/02_asyncio_event_flag.py` : `asyncio Event` синхронизация
- `examples/index.py` : скрипт за листване на всички примери на борда

Какво още не е потвърдено на реален борд

Книгата и примерите са прегледани за пълност и са подредени спрямо плана.

, Жe не e потвърдено пълно runtime изпълнение на всички примери върху реален VK_RA4M2.

- ай-чувствителните места остават:
- външните I2C устройства
- SPI `loopback` и външните SPI модули
- 7-сегментната индикация и матричната клавиатура
- `write` функциите на Flash и `dataflash`
- `lightsleep` и `deepsleep`

Заклучение

Тази версия на книгата е построена като проверка за пълност:

- всички теми от плана са отразени
- всички 54 примерни файла са споменати изрично

- всяка голяма тема има поне един реален кодов откъс
- порт-специфичните ограничения на VK_RA4M2 са записани ясно
- ледважата разумна стъпка вече не е 'оЖе съдържание', а хардуерна валидация на примерите по групи върху реалната платка.